

Guide Complet - Développement d'Application iOS avec UIKit

Introduction

UIKit est le framework historique d'iOS pour créer des interfaces utilisateur. Bien qu'il ne soit pas la technologie la plus moderne (remplacé progressivement par SwiftUI), il reste indispensable pour comprendre le fonctionnement fondamental d'iOS et est encore largement utilisé dans l'industrie.

Ce guide vous accompagnera dans la création d'une application de consultation de documents, permettant de lister des fichiers et de les ouvrir dans une vue détaillée.

Objectifs du Projet

- Créer une application iOS native avec UIKit
- Implémenter une liste de documents
- Ajouter une fonctionnalité d'ouverture de fichiers depuis l'appareil
- Comprendre l'architecture MVC (Model-View-Controller)
- Maîtriser les composants essentiels d'UIKit

✂ Prérequis et Configuration

Environnement de Développement

- **Xcode 15+** (obligatoire pour le développement iOS)
- **macOS** (requis pour Xcode)
- **Simulateur iOS** (inclus avec Xcode)

Création du Projet

1. **Ouvrir Xcode**
2. **Créer un nouveau projet**
 - Choisir "App" dans la section iOS
 - Nom du projet : "Document App"
 - Interface : **Storyboard** (pas SwiftUI)
 - Langage : **Swift**
 - Ne pas utiliser le template "Document App" prédéfini

Structure du Projet iOS

Éléments Fondamentaux

1. Targets

Les targets définissent les différentes versions de votre application :

- **Target principal** : L'application elle-même
- **Test targets** : Pour les tests unitaires et d'interface
- Chaque target peut avoir ses propres paramètres de build, icônes, et configurations

2. Fichiers de Base

- **AppDelegate.swift** : Point d'entrée de l'application, gère le cycle de vie global
- **SceneDelegate.swift** : Gère les scènes d'interface (iOS 13+)
- **ViewController.swift** : Contrôleur de vue principal
- **Main.storyboard** : Interface graphique principale
- **Info.plist** : Configuration de l'application

3. Assets.xcassets

Dossier contenant toutes les ressources visuelles :

- Icônes d'application
- Images utilisées dans l'app
- Couleurs personnalisées
- Gestion automatique des différentes résolutions d'écran

4. Storyboard

Interface graphique visuelle permettant de :

- Concevoir l'interface utilisateur par drag & drop
- Définir les transitions entre écrans (segues)
- Configurer les contraintes de mise en page (Auto Layout)

5. Simulateur iOS

Émulateur d'appareils iOS permettant de :

- Tester l'application sans appareil physique
- Simuler différents modèles d'iPhone/iPad
- Tester différentes versions d'iOS



Raccourcis Xcode Essentiels

Raccourci	Action
Cmd + R	Compiler et lancer l'application

Raccourci	Action
Cmd + Shift + O	Ouvrir rapidement un fichier
Cmd + /	Commenter/Décommenter la sélection
Ctrl + I	Indentation automatique du code
Cmd + Shift + Y	Afficher/Masquer la console de debug
Cmd + 0	Afficher/Masquer le navigateur
Cmd + Option + 0	Afficher/Masquer l'inspecteur



Architecture de l'Application Document App

Fonctionnalités Principales

1. Liste de Documents

- Affichage des documents dans une table view
- Icônes différenciées selon le type de fichier
- Nom et informations des fichiers

2. Ajout de Documents

- Bouton pour importer des fichiers
- Sélection depuis les documents de l'appareil
- Support de différents formats de fichiers

3. Vue Détaillée

- Ouverture des documents sélectionnés
- Affichage adapté selon le type de fichier
- Navigation retour vers la liste

Architecture MVC

```

  Models/
  └── Document.swift ..... // Structure des données

  Views/
  ├── Main.storyboard ..... // Interface utilisateur
  └── DocumentCell.swift ..... // Cellule personnalisée (optionnel)

  Controllers/
  ├── DocumentListViewController.swift ... // Liste des documents
  └── DocumentDetailViewController.swift // Vue détaillée

```

Composants UIKit Essentiels

1. UIViewController

```
swift

class DocumentListViewController: UIViewController {
    ... override func viewDidLoad() {
        ... super.viewDidLoad()
        setupUI()
    }
    ...
    ... private func setupUI() {
        ... // Configuration de l'interface
    }
}
```

2. UITableView

```
swift

@IBOutlet weak var tableView: UITableView!

// Dans viewDidLoad()
tableView.dataSource = self
tableView.delegate = self
```

3. UINavigationController

- Gestion de la pile de navigation
- Barre de navigation automatique
- Boutons de retour intégrés

4. UIDocumentPickerViewController

```
swift

func presentDocumentPicker() {
    ... let documentPicker = UIDocumentPickerViewController(forOpeningContentTypes: [item])
    ... documentPicker.delegate = self
    ... present(documentPicker, animated: true)
}
```

Modèle de Données

Structure Document

```
swift

struct Document {
    ... let name: String
    ... let url: URL
    ... let type: String
    ... let size: Int64
    ... let dateModified: Date
    ...
    ... var formattedSize: String {
    ...     ByteCountFormatter.string(fromByteCount: size, countStyle: .file)
    ... }
}
```

Flux de Navigation

1. **Écran Principal** : Liste des documents
2. **Action Utilisateur** : Tap sur un document OU tap sur le bouton "+"
3. **Navigation** :
 - Si tap sur document → Vue détaillée
 - Si tap sur "+" → Document Picker
4. **Retour** : Bouton de navigation ou geste

Bonnes Pratiques

Code Swift

- **Paramètres nommés** dans toutes les fonctions

```
swift

func addDocument(named fileName: String, from url: URL) {
    ... // Implementation
}
```

Interface Utilisateur

- **Auto Layout** pour l'adaptabilité multi-écrans
- **Safe Areas** pour éviter les zones système
- **Accessibilité** avec les labels appropriés

Architecture

- **Séparation MVC** claire
- **Delegates et DataSources** pour UITableView
- **Extensions** pour organiser le code

Étapes de Développement

Phase 1 : Configuration

1. Créer le projet Xcode
2. Configurer le storyboard principal
3. Créer les ViewControllers

Phase 2 : Interface Utilisateur

1. Concevoir la liste avec UITableView
2. Ajouter le bouton d'ajout
3. Créer la vue détaillée

Phase 3 : Logique Métier

1. Implémenter le modèle Document
2. Gérer les données avec un tableau
3. Connecter les actions utilisateur

Phase 4 : Fonctionnalités Avancées

1. Document Picker pour import
2. Persistance des données (optionnel)
3. Gestion des erreurs

Phase 5 : Tests et Finalisation

1. Tests sur simulateur
2. Tests sur différents appareils
3. Optimisations et corrections

Points Clés à Retenir

- **UIKit reste fondamental** pour comprendre iOS
- **MVC** est l'architecture de base d'iOS
- **Storyboard** facilite la conception d'interface

- **Auto Layout** assure l'adaptabilité
- **Delegates et DataSources** sont des patterns essentiels
- **La documentation Apple** est votre meilleure ressource

Ressources Complémentaires

- [Documentation officielle UIKit](#)
 - [Guide Human Interface Guidelines](#)
 - [Swift Language Guide](#)
-

Ce guide constitue une base solide pour débiter avec UIKit. L'apprentissage se fait par la pratique : n'hésitez pas à expérimenter et à poser des questions !